



Tecnológico de Monterrey

Inteligencia artificial avanzada para la ciencia de datos (Gpo 601)

Módulo 2 Uso de framework o biblioteca de aprendizaje máquina para la implementación de una solución. (Portafolio Implementación)

Carlos Manuel Delgado Vázquez

Matrícula: A01747037

IDM

Repositorio: https://github.com/CDV-97/M2_ML.git

Objetivo

El objetivo de este proyecto fue implementar una solución de machine learning utilizando una biblioteca especializada, demostrando conocimiento sobre el uso y configuración del framework.

Se utilizó un **Árbol de Decisión para clasificación** mediante DecisionTreeClassifier de la biblioteca scikit-learn. El modelo fue aplicado al dataset **Water Potability**, con el propósito de clasificar muestras de agua como potables o no potables.

Dataset utilizado

Se utilizó el dataset **Water Potability**, obtenido de Kaggle, contiene 3,276 observaciones y 10 columnas, incluyendo la variable objetivo.

El objetivo del dataset es clasificar muestras de agua según su potabilidad:

- 0: agua no potable
- 1: agua potable

Las variables utilizadas fueron:

- ph
- Hardness
- Solids
- Chloramines
- Sulfate
- Conductivity
- Organic_carbon
- Trihalomethanes
- Turbidity

La variable objetivo fue:

- Potability

El dataset contiene algunos valores faltantes. Para poder utilizar todas las observaciones, los valores faltantes fueron reemplazados con la media de cada variable.

División de entrenamiento y prueba

El conjunto de datos fue dividido en:

- **80% para entrenamiento**
- **20% para prueba**

La división se realizó con:

```
None
train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Se utilizó `random_state = 42` para obtener resultados reproducibles. También se utilizó `stratify = y` para mantener aproximadamente la misma proporción de las clases 0 y 1 en los conjuntos de entrenamiento y prueba. El conjunto de prueba estuvo compuesto por **656 observaciones**.

Framework utilizado

Para la implementación del modelo se utilizó la biblioteca:

```
None
skikit-learn
```

Específicamente:

None

```
from sklearn.tree import DecisionTreeClassifier
```

A diferencia de la implementación sin framework, en esta versión la estructura y lógica interna del Árbol de Decisión son proporcionadas por scikit-learn. El objetivo de esta implementación fue demostrar el uso correcto del framework y la configuración de los principales parámetros del algoritmo.

Configuración del Árbol de Decisión

El modelo utilizado fue configurado de la siguiente manera:

None

```
model = DecisionTreeClassifier(  
    criterion="gini",  
    max_depth=3,  
    min_samples_split=2,  
    random_state=42  
)
```

criterion="gini" : Indica que el árbol utiliza la **impureza Gini** para evaluar la calidad de las divisiones. El algoritmo busca divisiones que reduzcan la mezcla entre las distintas clases.

max_depth=3 : Limita la profundidad máxima del árbol a tres niveles. Esto ayuda a controlar la complejidad del modelo y evita que el árbol crezca de manera excesiva.

min_samples_split=2 : Indica que un nodo necesita al menos dos observaciones para poder considerar una nueva división.

random_state=42 : Permite mantener reproducibilidad en los resultados obtenidos.

Entrenamiento y predicción

El modelo fue entrenado utilizando:

```
None  
model.fit(X_train, y_train)
```

Posteriormente, las predicciones sobre el conjunto de prueba fueron generadas mediante:

```
None  
predictions = model.predict(X_test)
```

También se imprimieron ejemplos individuales de predicción para comprobar directamente el funcionamiento del modelo.

```
Ejemplos de predicciones:  
Ejemplo 1: Real = 1, Predicción = 0  
Ejemplo 2: Real = 0, Predicción = 0  
Ejemplo 3: Real = 0, Predicción = 0  
Ejemplo 4: Real = 0, Predicción = 0  
Ejemplo 5: Real = 0, Predicción = 0
```

Métrica principal seleccionada

La métrica principal seleccionada fue el **F1-score de la clase potable (1)**.

El F1-score combina dos métricas:

- **Precision:** proporción de predicciones positivas que fueron correctas.
- **Recall:** proporción de casos positivos reales que fueron identificados correctamente.

La fórmula es:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

Se eligió F1-score porque permite evaluar de forma conjunta la capacidad del modelo para identificar correctamente la clase positiva. El accuracy también fue utilizado como medida general, pero por sí solo puede ocultar diferencias importantes en el rendimiento entre las dos clases.

Prueba de ejecución

El proyecto puede ejecutarse directamente mediante:

```
Python
con_framework/main.py
```

Al ejecutarse, el programa carga el dataset, realiza el preprocesamiento, divide los datos en entrenamiento y prueba, entrena el Árbol de Decisión, genera predicciones y calcula las métricas de evaluación.

```
Ejemplos de predicciones:
Ejemplo 1: Real = 1, Predicción = 0
Ejemplo 2: Real = 0, Predicción = 0
Ejemplo 3: Real = 0, Predicción = 0
Ejemplo 4: Real = 0, Predicción = 0
Ejemplo 5: Real = 0, Predicción = 0

Accuracy:
0.6326219512195121

F1-score de la clase potable:
0.25386996904024767

Matriz de confusión:
[[374  26]
 [215  41]]

Reporte de clasificación:
      precision    recall  f1-score   support

      0       0.63       0.94       0.76       400
      1       0.61       0.16       0.25       256

 accuracy          0.63
 macro avg          0.62
weighted avg          0.63
```

Resultados

Accuracy

El modelo obtuvo un accuracy aproximado de:

$$\text{Accuracy} = 0.6326$$

Es decir, aproximadamente 63.2% de las observaciones del conjunto de prueba fueron clasificadas correctamente.

F1-score de la clase potable

El resultado obtenido fue aproximadamente:

F1=0.254

Matriz de confusión

La matriz de confusión obtenida fue:

Python

```
[ [374 26]  
  [215 41]]
```

Interpretación:

- **374** muestras de clase 0 fueron correctamente clasificadas como no potables.
- **26** muestras de clase 0 fueron clasificadas incorrectamente como potables.
- **215** muestras de clase 1 fueron clasificadas incorrectamente como no potables.
- **41** muestras de clase 1 fueron correctamente clasificadas como potables.

Métricas de clasificación

Los resultados por clase fueron aproximadamente:

Clase	Precision	Recall	F1-score
0	0.63	0.94	0.76
1	0.61	0.16	0.25

Para la clase 0, el modelo presenta un recall alto de **0.94**, por lo que logra identificar correctamente la mayoría de las muestras no potables.

Para la clase 1, el recall es de **0.16**, lo que significa que el modelo tiene mayor dificultad para detectar correctamente las muestras de agua potable.

Análisis de resultados

El Árbol de Decisión implementado mediante scikit-learn funciona correctamente y es capaz de generar predicciones sobre datos que no fueron utilizados durante el entrenamiento. El modelo presenta un desempeño notablemente mejor para la clase de agua no potable que para la clase potable. Aunque el accuracy general es de aproximadamente 63.26%, esta métrica no muestra por sí sola la diferencia existente entre las dos clases.

Por esta razón se seleccionó el F1-score de la clase potable como métrica principal, ya que permite considerar simultáneamente precision y recall. El resultado obtenido muestra que el modelo tiene una capacidad limitada para detectar correctamente la clase potable, pero esto no impide demostrar el funcionamiento adecuado del algoritmo y el uso correcto del framework.

Conclusión

Se logró implementar correctamente un Árbol de Decisión utilizando scikit-learn, configurando parámetros como criterion, max_depth, min_samples_split y random_state, y evaluando su desempeño mediante distintas métricas. El modelo obtuvo aproximadamente 63.26% de accuracy y un F1-score de 0.25 para la clase potable.

Un resultado relevante de esta implementación es que las métricas obtenidas fueron prácticamente idénticas a las obtenidas con el Árbol de Decisión implementado manualmente sin framework. Esto sugiere que la implementación desarrollada anteriormente reprodujo de forma correcta el comportamiento básico de un Árbol de Decisión bajo una configuración equivalente.

Aunque el desempeño predictivo del modelo es limitado, el objetivo principal de la actividad fue cumplido: utilizar correctamente un framework de Machine Learning, configurar el algoritmo, generar predicciones y evaluar sus resultados mediante una matriz de confusión y métricas apropiadas.

Link del Dataset

<https://www.kaggle.com/datasets/adityakadiwal/water-potability>